Linux 2

Cos'è uno script

Uno script è una serie di comandi di shell scritti in un file di testo.
Sono stati introdotti alcuni costrutti di controllo tipici di un vero linguaggio di programmazione per rendere gli script di shell più potenti.
Perché usare gli script di shell?

- **Per creare nuovi comandi personalizzati**
- **Per risparmiare tempo (si esegue uno script, invece di molti comandi in sequenza)**
- **Per automatizzare dei task che devono essere eseguiti frequentemente**
- **Per facilitare i compiti dell'amministratore di sistema.**
  Come scrivere uno script?
  Si può utilizzare un qualunque editor di testo (per esempio nano).

Scrivere uno script

Si provi a scrivere lo script `hello_world.sh`
Per fare ciò:

1. Si crea la directory in cui verranno salvati i nostri script

Shell

```
$ mkdir scripts
```

2. Ci si sposta nella directory *scripts* appena creata

Shell

```
$ cd ./scripts
```

3. Si apre l'editor di testo e si crea un file `hello_world.sh`

Shell

```
$ nano hello_world.sh
```

Shell di esecuzione

La prima riga di uno script può essere utilizzata per indicare quale shell intrepreta lo script.

- Per indicare uno script per la shell bash:

Shell

```shell
#! /bin/bash
```

Per conoscere quale shell si sta utilizzando, si utilizza il comando:

Shell

```shell
$ echo $SHELL
```

I commenti

I commenti vengono indicati con il simbolo `#`, e vengono ignorati dal interprete.

Attenzione

`#!` e `#` sono differenti, in quanto:

- `#!` :
  - è una sequenza di caratteri speciale ed indica al sistema operativo quale interprete utilizzare per eseguire lo script.
  - Deve essere inserito nella prima riga di uno script per poter essere riconosciuto correttamente
- `#` :
  - è un simbolo utilizzato per indicare un commento all'interno di uno script.
  - I commenti vengono ignorati dall'interprete dello script.
  - Questo simbolo può essere usato in qualsiasi riga dello script per aggiungere notazioni o spiegazioni.

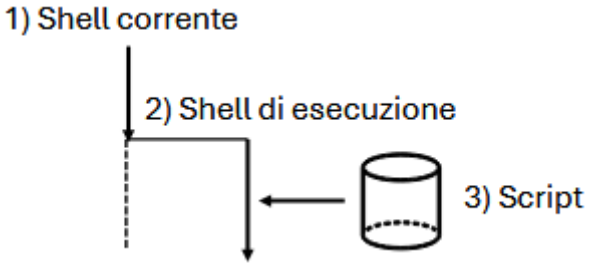
Esecuzione di uno script

Per eseguire uno script, è possibile utilizzare:

1. `$ bash < nome_script.sh` :
   - re-direzione dello standard di input sullo script
2. `$ bash nome_script.sh`
3. `$ ./nome_script.sh`

La shell riconosce che il file di input è uno script e determina quale shell deve essere utilizzata per eseguirlo.

- La shell che esegue lo script, lo esegue leggendo i comandi come se fossero digitati direttamente nella shell stessa.



Esecuzione di uno script.png

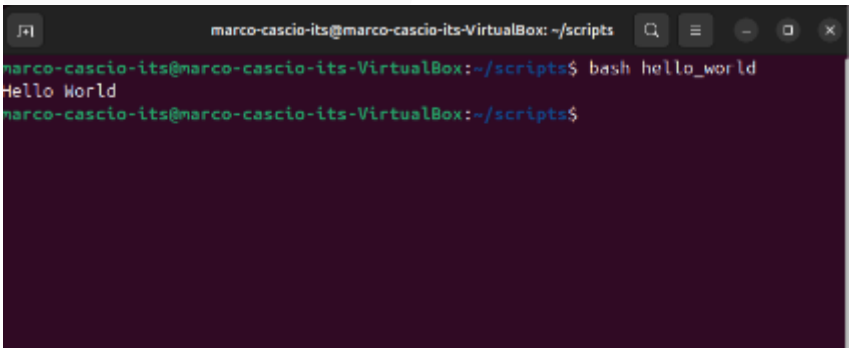
Esempi concreti

Per fare degli esempi concreti:

1. `$ bash < nome_script.sh`

Esempio modo 1 per l'esecuzione di uno script.png

2. `$ bash nome_script.sh`



Esempio modo 2 per l'esecuzione di uno script.png

3. `$ ./nome_script.sh`
   Se scriviamo `./hello_world.sh`, l'output sarà: *Permission denied*, questo perché bisogna definire i permessi:

Shell

```shell
$ chmod u+x hello_world.sh
```

- In questo modo si sta assegnando all'utente il permesso di esecuzione dello script `hello_world.sh`

Variabili di shell

Come per i linguaggi di programmazione una variabile è:
un nome simbolico al quale è associato un valore.
Sono utilizzate per memorizzare dati che possono essere utilizzati all'interno di script e sessioni di shell.

La shell permette di definire 2 tipi di variabili:

1. **variabili locali**
2. **variabili di ambiente**

Variabili locali

Le variabili locali non richiedono alcun comando speciale per essere create, sono semplicemente definite con un'assegnazione di valore.
Inoltre hanno una validità limitata all'ambito della shell stessa, in quanto sono visibili ed esistono solo all'interno e durante l'esecuzione del contesto in cui sono state definite, come una funzione o uno script.
Il nome di una variabile può contenere lettere, cifre e underscore.

ma il primo carattere non può essere un numero.

La creazione di una variabile e l'assegnazione di un valore si ottengono con una dichiarazione del tipo:

Shell

```
nome_variabile=valore #senza spazi.
```

L'assegnazione dei valori per queste variabili seguono le seguenti regole:

- **Se non viene fornito il valore da assegnare, si intende che la variabile è uguale ad una stringa vuota**
- **Se la variabile non esiste, allora viene creata con il valore specificato**.
- **Se la variabile esiste, il suo valore precedente viene sovrascritto**.

Il comando `unset` e il comando `$`

Il comando `unset`:

- permette di cancellare una variabile locale.

Shell

```
$ unset nome_variabile
```

Mentre il simbolo `$`:

- **permette di accedere al valore di una variabile**.

Shell

```
$ $nome_variabile
```

Esempi di casi d'uso

- Creare la variabile locale con valore `ciao`

Shell

```
$ a=ciao
```

- Visualizzare/accedere al valore della variabile `a`

```shell
$ echo $a
```

- Definire la variabile locale `b` con valore il comando `ls`

```shell
$ b=ls
```

- Sovrascrivere la variabile locale `b` con valore il comando `who`

```bash
$ b=who
```

- Visualizzare/accedere al valore della variabile `b`

```shell
$ echo $b
```

- Eseguire il comando `$ who`

```shell
$ $b
```

Variabili di ambiente

Le variabili di ambiente sono visibili a tutti i processi figli della shell. **Questo significa che quando un processo viene eseguito dalla shell, erediterà le variabili di ambiente definite nella shell genitore**. **Le variabili di ambiente persistono e sono accessibili ai processi figli finché la shell genitore è attiva o la variabile non viene esplicitamente eliminata**. Esiste un insieme di variabili di ambiente predefinite, che contengono informazioni utili sull'ambiente di esecuzione. Ad esempio:

- `$HOME` : **path della home directory**.

- `$PATH` : **path delle directory dove la shell, dopo l'inserimento di un comando, cerca il programma da eseguire**.
- `$MAIL` : **path della mailbox dell'utente**.
- `$USER` : **username dell'utente**.
- `$SHELL` : **path della shell di login**.
- `$TERM` : **specifica il tipo di terminale utilizzato**.

Il comando `read`

legge dallo standard input ed assegna alla prima variabile specificata la prima parola digitata, alla seconda variabile specificata la seconda parola digitata e così via.

Shell

```shell
echo "Scrivi il tuo nome e cognome"
#usa il comando read per assegnare alle variabili name e surname i valori di nome e cognome digitati
read name surname
echo "Ciao, $name $surname"
```

il comando `$bash` serve per eseguire i file:
`$ bash read_script.sh` : esegue lo script `read_script.sh`

Shell

```shell
bash read_script
```

Operazioni aritmetiche

In linux è possibile eseguire operazioni aritmetiche tramite i seguenti operatori:

- `+` : somma
- `-` : sottrazione
- `*` :prodotto
- `/` : divisione intera
- `%` : resto

Mentre gli operatori relazionali comuni sono:

Operatore positivo	Significato	Operatore negativo	Significato
`>`	maggiore	`<`	minore
`>=`	maggiore o uguale	`<=`	minore o uguale
`=`	uguale	`!=`	diverso
`&&`	AND Logico	`||`	`OR logico`

Il comando `$ expr`

Sta per <*espressioni aritmetiche*>:

- **permette di usare variabili numeriche e di eseguire calcoli semplici.**

Shell

```
$ expr 1 + 2 #attenzione agli spazi
#Output: 3
```

- In questo caso effettua la somma tra `1` e `2`

Shell

```
$ a = 9 b=3 #definisce due variabili numeriche
$expr $a - $b #effetua la sottrazione tra a e b
```

Si possono anche usare le **parentesi tonde e quadre** per rappresentare le espressioni aritmetiche:

Shell

```
$ a = 9 b=3 #definisce due variabili numeriche
$ echo $($a+$b)
#Output: 12
```

Oppure:

Shell

```
$ a = 9 b=3 #definisce due variabili numeriche
$ echo $[$a-$b]
```

Strutture di controllo

In ambiente **Linux**, come in qualunque linguaggio di programmazione, le **strutture di controllo** permettono di **decidere quali istruzioni eseguire** in base al verificarsi di determinate condizioni.

Le principali strutture di controllo utilizzabili nella **Shell** sono:

- `if`
- `case`
- `for`
- `while`

Struttura dell' if

La struttura dell' `if` in shell programming è concettualmente simile a quella di altri linguaggi, come in python
La sintassi generale è la seguente:

Shell

```
if <condizione>
then
    <comandi_se_vera>
else
    <comandi_se_falsa>
fi

```

in Bash, il blocco `if` termina sempre con la parola chiave `fi` (ovvero "if" scritto al contrario).

Se la **condizione** (o lista di comandi) restituisce **successo** (cioè un valore di uscita uguale a 0), vengono eseguiti i comandi presenti dopo `then`.
In caso contrario, vengono eseguiti quelli indicati dopo `else`.

Operatori utilizzabili nelle condizioni

Operatori matematici

Quando si confrontano valori numerici, si utilizzano i seguenti operatori:

Operatore	Significato
`-eq`	uguale a
`-ne`	diverso da
`-lt`	minore di
`-le`	minore o uguale a
`-gt`	maggiore di
`-ge`	maggiore o uguale a

Esempio:

Shell

```shell
if [ $a -le $b ]
then
    echo "a è minore o uguale a b"
else
    echo "a è maggiore di b"
fi

```

Operatori logici

Gli operatori logici permettono di combinare più condizioni:

Operatore	Significato
`!expr`	NOT (nega l'espressione)
`expr1 -a expr2`	AND (entrambe le condizioni devono essere vere)
`expr1 -o expr2`	OR (almeno una delle due condizioni deve essere vera)

Esempio:

Shell

```shell
if [ $a -gt 0 -a $b -gt 0 ]
then
```

```
    echo "Entrambi i numeri sono positivi"
fi
```

Operatori su file

Questi operatori consentono di verificare proprietà relative a file o directory.

Operatore	Verifica se...
`-s file`	Il file ha dimensione superiori a 0
`-f file`	Il file esiste e non è una directory
`-d dir`	La directory esiste
`-w file`	Il file è scrivibile
`-r file`	Il file è leggibile
`-x file`	Il file è eseguibile

Esempio:

Shell

```
if [ -d scripts ]
then
    echo "La directory 'scripts' esiste"
else
    echo "La directory 'scripts' non esiste"
fi
```

Il comando `test`

Il comando `test` viene utilizzato per **valutare una condizione** e restituire un valore di verità (vero/falso).
La sua forma generale è:

Shell

```
test <condizione>
```

Oppure, in forma equivalente e più comune:

Shell

```
[ <condizione> ]
```

Esempi:

```shell
test -d scripts        # verifica se "scripts" è una directory
test $a -le $b         # verifica se a è minore o uguale a b
```

oppure equivalenti con le parentesi quadre:

```shell
[ -d scripts ]
[ $a -le $b ]
```

Esercizio 1— Verifica di file e directory

Scrivere uno script denominato `file_dir.sh` che accetti **due parametri** in ingresso:

1. il **nome di un file**,
2. il **nome di una directory**.

Lo script deve **verificare se il file e la directory esistono** e, in caso affermativo, **visualizzare un messaggio appropriato** in output.

Soluzione passo per passo:

1. Creare il file dello script:

```shell
$ nano file_dir.sh
```

2. Scrivere al suo interno il seguente codice:

```shell
#! /bin/bash
# Script: file_dir.sh
# Scopo: Verificare se un file e una directory esistono

# Controlla se il primo argomento ($1) è un file esistente
```

```
if test -f $1
then
    echo "$1 esiste ed è un file"
else
    echo "$1 non esiste oppure non è un file"
fi

# Controlla se il secondo argomento ($2) è una directory esistente
if test -d $2
then
    echo "$2 esiste ed è una directory"
else
    echo "$2 non esiste oppure non è una directory"
fi

```

3. Rendere lo script eseguibile:

Shell

```
$ chmod +x file_dir.sh
```

4. Creare un file di prova:

Shell

```
$ touch my_file.txt
```

5. Eseguire lo script:

Shell

```
$ bash file_dir.sh my_file.txt my_dir
```

Output previsto:

```
my_file.txt esiste ed è un file
my_dir non esiste oppure non è una directory
```

6. Creare la directory e rieseguire lo script:

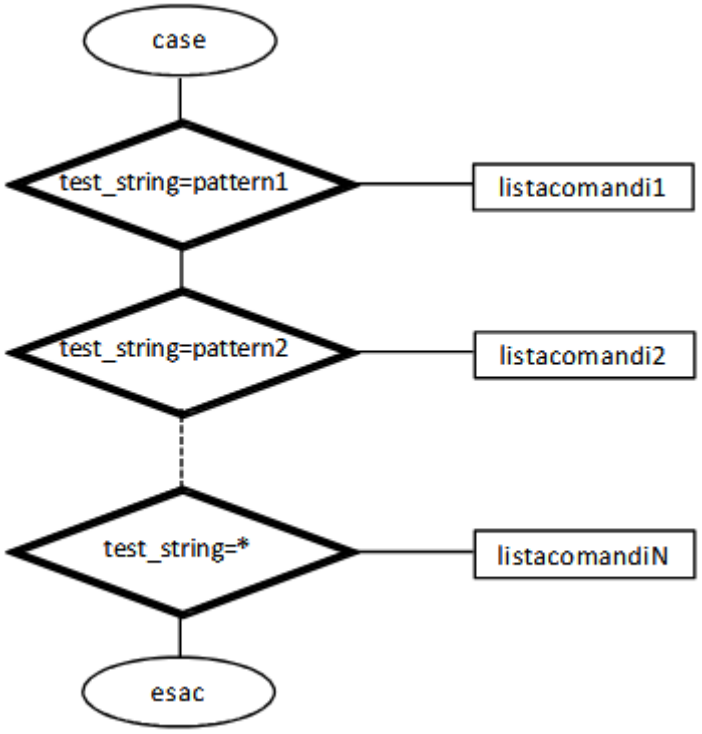
```shell
$ mkdir my_dir
$ bash file_dir.sh my_file.txt my_dir
```

Output aggiornato:

```shell
my_file.txt esiste ed è un file
my_dir esiste ed è una directory
```

Struttura case

La struttura `case` in Bash consente di **scegliere** quale blocco di comandi eseguire tra diversi possibili, in base al **valore di una stringa**.
È l'equivalente della struttura `switch` presente in altri linguaggi di programmazione (come C o JavaScript).



Struttura case.png

La sintassi generale è la seguente:

```shell
case <test_string> in
    pattern1 ) <lista_comandi1> ;;
    pattern2 ) <lista_comandi2> ;;
```

```
    ...
    * ) <lista_comandi_default> ;;
esac
```

la parola chiave `esac` chiude la struttura, ed è semplicemente `case` scritto al contrario.

Funzionamento

- Il valore di `<test_string>` viene confrontato, **nell'ordine**, con ciascun **pattern** indicato.
- Quando viene trovata una **corrispondenza**, viene eseguita la lista di comandi associata a quel pattern.
- Se **nessun pattern corrisponde**, viene eseguita la lista di comandi del **caso predefinito** (`*`).

Esempio pratico

Shell

```shell
#!/bin/bash
# Script: giorno.sh
# Scopo: Stampare un messaggio in base al giorno della settimana

echo "Inserisci un giorno della settimana:"
read giorno

case $giorno in
    lunedi | martedi | mercoledi | giovedi | venerdi )
        echo "È un giorno lavorativo." ;;
    sabato | domenica )
        echo "È il weekend!" ;;
    * )
        echo "Valore non valido." ;;
esac
```

Esecuzione:

Shell

```shell
$ bash giorno.sh
Inserisci un giorno della settimana:
```

```
sabato
È il weekend!
```

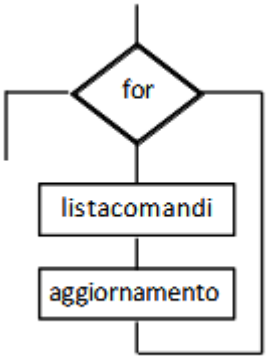
In sintesi:

- `case` è utile quando si devono gestire **più condizioni alternative**.
- Ogni **pattern** termina con `)` e il relativo blocco di comandi si conclude con `;;`.
- Il pattern `*` serve come **caso di default**, per gestire input non previsti.
- La struttura viene chiusa con `esac`.

Struttura `for`

La struttura `for` consente di **iterare** su una serie di elementi, eseguendo per ciascuno di essi una lista di comandi.

Ad ogni iterazione, la variabile assunta nel ciclo assume il valore di uno degli elementi della lista.



Struttura for.png

Shell

```shell
for i in word1 word2 ...
do
    <lista_comandi>
done
```

La parole chiave `done` chiude la struttura del loop for

Funzionamento

- Alla variabile `i` vengono assegnati, **uno alla volta**, i valori indicati dopo la parola chiave `in` (`word1`, `word2`, ...).
- Per ogni valore, viene eseguita la **lista di comandi** compresa tra `do` e `done`.
- Quando tutti i valori sono stati elaborati, il ciclo termina.

Esempio pratico

Shell

```shell
#!/bin/bash
# Script: saluto.sh
# Scopo: Stampare un saluto per ogni nome nella lista

for nome in Marco Luca Anna
do
    echo "Ciao $nome!"
done

```

Esecuzione:

Shell

```shell
$ bash saluto.sh
Ciao Marco!
Ciao Luca!
Ciao Anna!
```

In sintesi:

- Il ciclo `for` serve per **ripetere operazioni su un insieme di valori**.
- La parola chiave `do` introduce il blocco di comandi da eseguire.
- Il ciclo si chiude sempre con la parola chiave `done`.

Struttura `while`

La struttura `while` permette di ripetere un blocco di comandi finché una condizione è vera.
Finché la condizione (o lista di comandi) valutata restituisce **successo**, il ciclo continua; quando la condizione diventa falsa, il ciclo termina.

Shell

```shell
while <condizione_o_comando>
do
    <lista_comandi>
done
```

Come per il `for`, anche per il `while` la parola chiave `done` chiude la struttura.

Funzionamento

- La **condizione** viene valutata **all'inizio di ogni iterazione**.
- Se la condizione è **vera**, vengono eseguiti i comandi all'interno del blocco `do … done`.
- Quando la condizione diventa **falsa**, il ciclo termina e l'esecuzione continua con le istruzioni successive al ciclo.

Esempio pratico:

Shell

```shell
#!/bin/bash
# Script: countdown.sh
# Scopo: contare da 5 a 1

i=5
while [ $i -gt 0 ]
do
    echo "Countdown: $i"
    i=$((i-1))
done
echo "Lancio!"
```

Esecuzione:

Shell

```shell
$ bash countdown.sh
Countdown: 5
Countdown: 4
Countdown: 3
Countdown: 2
```

```
Countdown: 1
Lancio!
```

In sintesi:

- Il ciclo `while` è ideale quando **non si conosce a priori il numero di iterazioni**.
- La condizione viene controllata **prima di ogni iterazione**.
- Il ciclo termina automaticamente quando la condizione diventa falsa.